

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programación Dinámica

4 de mayo de 2026

Alejandro, Pablo Martin
98021

Prada, Matias Leonel
99397

Poli Salvador, Ignacio Manuel
110399

1. Consideraciones

En este trabajo practico, vamos a estar resolviendo el problema de Scaloni para definir los días que deba entrenarse y los días que convenga descansar de tal forma de tener la mayor ganancia posible para los posibles N entrenamientos.

Para realizarlo, definimos la siguiente nomenclatura:

- El esfuerzo que demanda el día i -ésimo es e_i , a su vez, esta es la ganancia máxima para ese día.
- La energía por día consecutivo disponible en el día i -ésimo viene definida por s_i
- Para cada día s_i (con $1 \leq i \leq N$) : $s_1 \geq s_2 \geq s_3 \geq \dots \geq s_N$.
- Tenemos N días de de esfuerzo al igual que tenemos N días de entrenamiento consecutivos.

Ejemplo de Resolución

2. Análisis del Problema

El problema presenta una serie de decisiones secuenciales donde cada día i el equipo técnico debe elegir entre dos opciones mutuamente excluyentes: **entrenar** o **descansar**.

Sabemos que la ganancia de un entrenamiento no es estática, sino que depende directamente de la fatiga acumulada del plantel. Podemos expresar la ganancia de entrenar un día específico j , habiendo entrenado ininterrumpidamente desde un día i (es decir, llevando $j - i + 1$ días de fatiga), mediante la siguiente expresión matemática:

$$g(i, j) = \text{mín}(e_j, s_{j-i+1}) \quad (1)$$

en donde e_j es la expectativa de esfuerzo del día j , y s representa el arreglo de energía disponible del equipo, el cual decrece a medida que aumentan los días consecutivos de entrenamiento.

2.1. Matriz de Ganancias

Para evitar recalcular las sumatorias de energía repetidas veces y aislar el concepto de “días consecutivos”, introducimos una matriz de ganancias $\text{Ganancias}(i, j)$. Esta estructura almacena la ganancia total obtenida si el equipo entrena de forma ininterrumpida desde el día i hasta el día j .

Debido a que la ganancia es acumulativa, podemos definir la construcción de esta matriz de manera iterativa:

$$\text{Ganancias}(i, j) = \begin{cases} 0 & \text{si } j < i \\ \text{Ganancias}(i, j-1) + \text{mín}(e_j, s_{j-i+1}) & \text{si } j \geq i \end{cases} \quad (2)$$

Esta precomputación resulta en una matriz triangular superior que se puede construir en tiempo $\mathcal{O}(n^2)$, y nos permite consultar la ganancia de cualquier racha de entrenamientos en tiempo constante $\mathcal{O}(1)$.

2.2. Ejemplo de la Matriz de Ganancias

Para ilustrar la construcción y el significado de la matriz $\text{Ganancias}(i, j)$, consideremos un caso de ejemplo con $n = 4$ días, donde los arreglos de esfuerzo esperado y energía disponible son:

- Esfuerzo (e): [10, 5, 12, 9]
- Energía (s): [15, 7, 5, 2]

En la estructura de esta matriz:

- Las **filas** (i) representan el **día de inicio** del bloque de entrenamiento ininterrumpido.
- Las **columnas** (j) representan el **día de finalización** de ese mismo bloque.
- Cada **celda** contiene la ganancia total acumulada si el equipo sale a entrenar todos los días en ese rango $[i, j]$.

Calculamos los primeros valores paso a paso aplicando la ecuación (2):

- **Fila 1 (Entrenamientos que empiezan el Día 1):** Arrancamos con la energía al máximo ($s_1 = 15$).

- $j = 1$ (Entrenar solo D1): $\min(e_1, s_1) = \min(10, 15) = 10$
 - $j = 2$ (Entrenar D1 y D2): $\text{Ganancias}(1, 1) + \min(e_2, s_2) = 10 + \min(5, 7) = 10 + 5 = 15$
 - $j = 3$ (Entrenar D1 al D3): $\text{Ganancias}(1, 2) + \min(e_3, s_3) = 15 + \min(12, 5) = 15 + 5 = 20$
- **Fila 2 (Entrenamientos que empiezan el Día 2):** Como asumimos que empezó el Día 2, la fatiga se resetea y usamos s_1 para el primer día de este bloque.
- $j = 2$ (Entrenar solo D2): $\min(e_2, s_1) = \min(5, 15) = 5$
 - $j = 3$ (Entrenar D2 y D3): $\text{Ganancias}(2, 2) + \min(e_3, s_2) = 5 + \min(12, 7) = 5 + 7 = 12$

Al completar todos los cálculos, la matriz resultante queda de la siguiente manera:

Inicio (i) \ Fin (j)	$j = 1$	$j = 2$	$j = 3$	$j = 4$
$i = 1$	10	15	20	22
$i = 2$	-	5	12	17
$i = 3$	-	-	12	19
$i = 4$	-	-	-	9

Cuadro 1: Matriz de Ganancias precomputada para $n = 4$.

Por ejemplo, si observamos la celda en la fila $i = 2$ y columna $j = 4$, el valor 17 representa explícitamente la ganancia obtenida si el equipo decide descansar el día 1, y luego entrena ininterrumpidamente durante los días 2, 3 y 4. Las celdas vacías debajo de la diagonal no son válidas matemáticamente, ya que un bloque de entrenamiento no puede terminar en un día anterior al que empezó.

2.3. Ecuación de Recurrencia (1D)

Con la matriz de ganancias definida, podemos modelar el problema principal utilizando Programación Dinámica. Definimos nuestro arreglo unidimensional de subproblemas $OPT(i)$ como “la ganancia máxima posible considerando únicamente los primeros i días del torneo”.

Para entender la lógica detrás de la ecuación, imaginemos que estamos parados en el día i y queremos evaluar cómo maximizar nuestra ganancia acumulada hasta hoy. En lugar de preguntarnos si hoy entrenamos o descansamos, el algoritmo cambia el enfoque y se pregunta: **¿De qué tamaño será la racha ininterrumpida de entrenamientos que finaliza exactamente hoy?**

Si llamamos k a la longitud de esa racha final de entrenamientos, el algoritmo evalúa exhaustivamente todas las opciones físicamente posibles (desde $k = 1$ hasta $k = i$) y se queda con la que otorgue el mayor puntaje. Al evaluar cada k , el problema se divide en dos escenarios:

- **Escenario de Racha Total ($k = i$):** Significa que la decisión a evaluar es si conviene que el equipo haya entrenado todos los días de corrido desde que empezó el torneo hasta el día de hoy, sin descansar nunca. Como no hay días previos, la ganancia de esta opción es directamente el bloque completo que ya precomputamos: $\text{Ganancias}(1, i)$.
- **Escenario de Racha Parcial ($1 \leq k < i$):** Significa que el equipo entrena los últimos k días consecutivos (desde el día $i - k + 1$ hasta el día i), pero **obligatoriamente descansó el día inmediatamente anterior** (el día $i - k$).

Al asumir que el día $i - k$ fue un día de descanso, sabemos que la fatiga del equipo se reseteó a cero. Por lo tanto, el puntaje total de esta opción será la suma de la ganancia de la nueva racha actual de k días, más la **mejor historia posible** que el equipo haya podido construir en los días previos a ese descanso.

Esa “mejor historia pasada” (que abarca desde el día 1 hasta el día $i - k - 1$) ya fue resuelta por el algoritmo en iteraciones anteriores y está guardada en $OPT(i - k - 1)$.

Al unificar esta lógica matemática, formalizamos la ecuación evaluando de forma independiente la racha total para evitar acceder a índices negativos o inválidos en el arreglo OPT :

$$OPT(i) = \max \begin{cases} \text{Ganancias}(1, i) & \text{(Racha total sin descansos)} \\ \max_{k \in \{1, \dots, i-1\}} \left(OPT(i-k-1) + \text{Ganancias}(i-k+1, i) \right) & \text{(Rachas parciales con descanso previo)} \end{cases}$$

2.4. Ejemplo del Cálculo de Óptimos (OPT)

Continuando con nuestro ejemplo de $n = 4$ días y utilizando la matriz de la Tabla 1, evaluaremos el arreglo de óptimos OPT aplicando la ecuación (3) de forma ascendente (*bottom-up*), recordando que asumimos $OPT(0) = 0$.

- **Día 1** ($i = 1$): La única opción válida es entrenar desde el inicio ($k = 1$).
 - $OPT(1) = \text{Ganancias}(1, 1) = 10$. (*Guardamos* $k = 1$).
- **Día 2** ($i = 2$): Evaluamos los posibles cortes k :
 - Entrenar D1 y D2 ($k = 2$): $\text{Ganancias}(1, 2) = 15$.
 - Descansar D1, entrenar D2 ($k = 1$): $OPT(0) + \text{Ganancias}(2, 2) = 0 + 5 = 5$.
 - El máximo es **15**. (*Guardamos* $k = 2$).
- **Día 3** ($i = 3$): Evaluamos los posibles cortes k :
 - Entrenar D1 al D3 ($k = 3$): $\text{Ganancias}(1, 3) = 20$.
 - Descansar D1, entrenar D2 y D3 ($k = 2$): $OPT(0) + \text{Ganancias}(2, 3) = 0 + 12 = 12$.
 - Descansar D2, entrenar D3 ($k = 1$): $OPT(1) + \text{Ganancias}(3, 3) = 10 + 12 = 22$.
 - El máximo es **22**. (*Guardamos* $k = 1$).
- **Día 4** ($i = 4$): Evaluamos los posibles cortes k :
 - Entrenar D1 al D4 ($k = 4$): $\text{Ganancias}(1, 4) = 22$.
 - Descansar D1, entrenar D2 al D4 ($k = 3$): $OPT(0) + \text{Ganancias}(2, 4) = 0 + 17 = 17$.
 - Descansar D2, entrenar D3 y D4 ($k = 2$): $OPT(1) + \text{Ganancias}(3, 4) = 10 + 19 = 29$.
 - Descansar D3, entrenar D4 ($k = 1$): $OPT(2) + \text{Ganancias}(4, 4) = 15 + 9 = 24$.
 - El máximo absoluto del problema es **29**. (*Guardamos* $k = 2$).

El arreglo final resulta ser $OPT = [10, 15, 22, 29]$. El valor $OPT(4) = 29$ representa la máxima ganancia posible para todo el torneo.

2.5. Complejidad y Reconstrucción

La solución planteada requiere dos fases. En primer lugar, la construcción de la matriz $\text{Ganancias}(i, j)$ toma tiempo $\mathcal{O}(n^2)$ y espacio $\mathcal{O}(n^2)$.

En segundo lugar, el cálculo del arreglo unidimensional OPT de tamaño n requiere iterar i desde 1 hasta n , y en el peor de los casos, el bucle interno de maximización evalúa $i - 1$ cortes posibles. Esto resulta en una sumatoria de Gauss, la cual también se acota en tiempo $\mathcal{O}(n^2)$.

El resultado óptimo del problema se encontrará en la última posición del arreglo, $OPT(n)$.

Para reducir la complejidad de reconstruir el cronograma de entrenamientos (camino óptimo), se va a almacenar el valor del k final para poder reconstruir cada paso en $\mathcal{O}(1)$, mejorando la reconstrucción a $\mathcal{O}(n)$. Basta con realizar un proceso de *backtracking* desde $OPT(n)$, con el último k obtenido, marcando esos k días como entrenamiento, el día anterior como descanso ($i - k$), y saltando recursivamente al estado del día anterior $OPT(i - k - 1)$, así sucesivamente hasta llegar al inicio (top-down).

3. Algoritmo Greedy

Sabemos que una solución Greedy divide el problema principal en sub-problemas o estados en los cuales se busca la mejor solución posible en cada uno de los estados y por consecuencia (si el algoritmo esta bien diseñado), se llega a la solución optima.

En nuestra caso, empecemos viendo que la formula (??) nos calcula el tiempo total para un caso de N rivales, sabemos que $\sum_{i=1}^N s_i$ va a ser un valor constante asique cualquier decisión que tomemos sobre dicho valores no va a modificar el tiempo total que vamos a ir calculando. Nuestra selección greedy tiene que ser sobre los tiempos de a_i aprovechando que tenemos N ayudantes para poder superponer los tiempos.

Teniendo en cuenta que los a_i conviene ponerlo en paralelo, la mejor selección que se puede hacer para reducir el tiempo total es comenzando con los tiempos mayores, así mientras un ayudante este mirando un rival, el siguiente ayudante puede continuar con el siguiente.

La siguiente figura muestra un caso con el ordenamiento

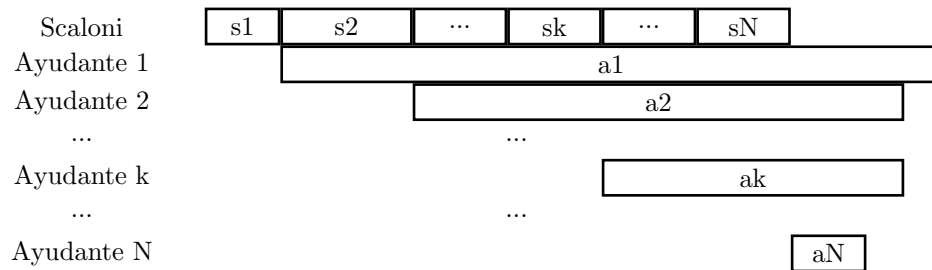


Figura 3.1: Solución Greedy propuesta.

Podemos ver que a_1 siendo el rival con mayor tiempo termina después de cada uno de los otros ayudantes. Por ende, el tiempo total es igual a

$$T = \sum_{i=1}^N s_i + (a_1 - \sum_{i=2}^N s_i) \quad (4)$$

Ahora, imaginemos un caso con el mismo conjunto en donde a_i comienza en una posicion k , de esta forma, el tiempo total seria el tiempo total seria

$$T = \sum_{i=1}^N s_i + (a_1 - \sum_{i=k}^N s_i) \quad (5)$$

haciendo que el tiempo total sea mucho mas grande debido a que al tiempo de a_1 se reduce con una cantidad menor de tiempos s_i , es decir, $a_1 - \sum_{i=2}^N s_i > a_1 - \sum_{i=k}^N s_i$.

```
1 TiempoTotal(rivales):  
2  
3   Ordenamos rivales por a_i del mas grande al mas chico  
4  
5   Sea T el tiempo total hasta el momento inicializado en 0.  
6   Sea a_maximo el ultimo a_i mas grande hasta el momento.  
7   Sea s_menos la suma de los s que hay que restar iniciado en 0.  
8  
9   Por cada r en rivales:  
10    T = T + r.s // sumamos el tiempo de scaloni  
11    // Si mi a_maximo es mas grande que mi s_menos mas r.s (scaloni actual) y el  
12    // r.a significa que el mi a_maximo termino despues  
13    Si r.a + s_menos + r.s > a_maximo:  
14        s_menos = 0  
15        a_maximo = r.a  
16    Sino:  
17        s_menos = s_menos + r.s  
18  
19    devolver T + a_maximo - s_menos, rivales ordenados  
20  
21
```

Figura 3.2: Pseudocódigo propuesto

tener en cuenta que el ejercicio pide la lista de los rivales ordenados por eso se devuelve dicho valor. En el calculo de tiempo máximo, se usaron las matemáticas explicadas antes para poder resolver el problema.

Las optimilidades del espacio y tiempo serian $O(n)$ y $O(n \log n)$ respectivamente, esto viene tomando el peor caso del algoritmo (los a_i están ordenados desde menor a mayor de entrada) que va a generar que el algoritmo de ordenamiento sea de $O(n \log n)$, y luego se hace una ciclo sobre la lista que da una complejidad de $O(n)$ haciendo que el algoritmo en total sea $O(n + n \log n) = O(n \log n)$. Para el espacio, solo se guarda toda la lista de rivales por lo que la complejidad es de $O(n)$.

Para la solución propuesta de ordenamiento se utilizó un algoritmo de **MergeSort** que nos da la solución en un orden de complejidad $O(n \log n)$

4. Análisis de Variabilidad y Optimalidad

A continuación, analizamos cómo la variabilidad de los parámetros de entrada afecta el comportamiento del algoritmo de Programación Dinámica, la optimalidad de la solución teórica y la posterior reconstrucción empírica del cronograma.

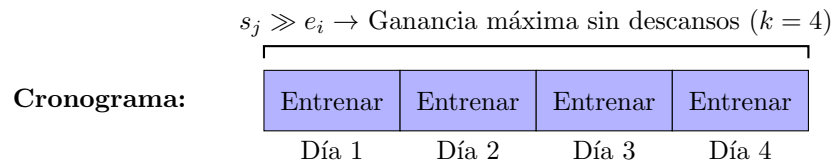
4.1. Afectación de la variabilidad a la ganancia y optimalidad

La variabilidad en las distribuciones de los valores de esfuerzo (e) y energía (s) **no afecta la optimalidad de la solución algorítmica**. Al evaluar exhaustivamente todos los tamaños de bloques posibles k hacia atrás y apoyarse en subestructuras óptimas ya calculadas, la ecuación de recurrencia garantiza matemáticamente encontrar siempre la ganancia máxima absoluta, sin importar si los datos son uniformes, extremos o atípicos.

Sin embargo, la forma de estos arreglos **sí afecta drásticamente la estructura del cronograma óptimo resultante** y la cantidad de soluciones óptimas posibles. Analizamos en detalle distintos escenarios:

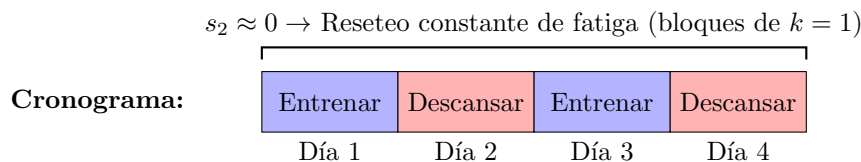
1. Caso A: Energía casi inagotable o constante.

- **Descripción:** El arreglo de energía s se mantiene en valores muy altos y decae sumamente lento, superando holgadamente a los valores de esfuerzo e_i en la gran mayoría de los días.
- **Análisis:** En este escenario, la función de ganancia diaria $\min(e_j, s_{j-i+1})$ estará dominada exclusivamente por los valores de e_j . Dado que la energía no es un limitante, tomar un día de descanso implica sumar 0 a la ganancia de ese día sin obtener un beneficio real a cambio para el futuro. Como resultado, el algoritmo tenderá a elegir bloques de tamaño máximo ($k = i$), construyendo un cronograma de entrenamiento continuo y sin interrupciones.



2. Caso B: Caída abrupta de energía.

- **Descripción:** El equipo sufre de fatiga extrema muy rápido. Por ejemplo, la energía inicial s_1 tiene un valor alto, pero para el segundo día consecutivo (s_2) decae abruptamente a valores cercanos a 0.
- **Análisis:** Si la energía se desploma al segundo día de entrenamiento, cualquier bloque de tamaño $k \geq 2$ sumará un valor marginal en sus últimos días. Al algoritmo le resultará matemáticamente mucho más rentable insertar un día de descanso para resetear la fatiga y volver a aprovechar el alto valor de s_1 . En consecuencia, la estructura del cronograma será una secuencia fragmentada, alternando estrictamente días de entrenamiento cortos (bloques de $k = 1$) con días de descanso.



4.2. Comportamiento en Programación Dinámica (Bottom-Up) vs. Reconstrucción (Top-Down)

En contextos de alta variabilidad que generan empates sistemáticos (como en el Caso C), el impacto se debe interpretar de manera distinta según la fase del algoritmo:

- **Fase Bottom-Up (Llenado del arreglo OPT):** La variabilidad y la multiplicidad de opciones **no afectan en lo absoluto el cálculo del valor óptimo**. Cuando la función máx de la ecuación de recurrencia evalúa diferentes tamaños de bloque k y descubre que varios devuelven el mismo puntaje máximo, la ecuación simplemente asimila ese número. El valor final acumulado que llegará a $OPT(n)$ será único, robusto y representará la ganancia teórica inamovible, a prueba de cualquier volatilidad en los datos de entrada.
- **Fase Top-Down (Algoritmo de Reconstrucción):** Aquí es donde la alta variabilidad tiene un impacto real y directo. Si durante la fase *Bottom-Up* existieron múltiples valores de k que empataron como el máximo para un mismo día i , significa que existen **múltiples cronogramas estructuralmente distintos que logran exactamente la misma ganancia total**.

Un algoritmo de reconstrucción típico almacena solo un valor de k ganador por celda para lograr una complejidad de $\mathcal{O}(n)$. La decisión de *cuál* k se guarda ante un empate depende de

un simple operador lógico en la implementación: si el código utiliza un comparador estricto ($>$), conservará el primer k que encuentre; si utiliza mayor o igual ($\geq$), sobrescribirá la variable y se quedará con el último k óptimo evaluado.

Por lo tanto, ante datos que generan empates constantes, el cronograma específico devuelto (el detalle de qué días puntuales se entrena o se descansa) puede variar diametralmente de una ejecución a otra o entre distintas implementaciones del mismo algoritmo, **aunque todas esas versiones representen caminos 100 % óptimos**.